CIRCVNCISION DE COMEDIAS AI Transcription Documentation

GIT HUB LINK

May 18 – Ewan McPhillamy – July 11

2025

Table Of Contents

Goal

Overview

After meeting with Prof. Sáenz Nuño and Prof. Puente Águeda, we decided that my goal would be to create a tool using AI to read the mostly typewritten, scanned-to-PDF book *CIRCVNCISION DE COMEDIAS*. The plan was to start by training a model based on data from the first chapter. I was provided with that chapter, and after some discussion, we realized that the time I had for this research was too short to handle everything—especially the margins, handwritten notes, and the main typewritten text. So, I decided to focus solely on the main text.

Since I had basically zero experience with AI, a lot of my time was spent just figuring out what was going on—how things worked, how to fix bugs, and how to keep things moving forward. This project has been both a learning exercise and the first step toward something bigger: a tool that can eventually transcribe the entire book.

To make that happen, I think the project needs to be broken into four main parts—each with its own challenges: deconstructing, transcribing, grammatical tweaking, and rebuilding.

Deconstructing

This step is all about splitting each page of the book into manageable pieces. These would probably fall into four categories: *margins*, *main text*, *titles*, and *handwritten notes*. These categories seem like the most useful ones to focus on, especially to avoid wasting time transcribing stuff we don't need.

The way I imagine this working is by training a model to detect and label each of those sections—probably by generating .box files for every page that define the location of each group. This part would probably be just as hard as the transcription part since it also involves model training.

Transcribing

Once the content is broken up, the next step is transcription. That means training an OCR (Optical Character Recognition) model to recognize the text in scanned images. Challenges here include everything from faded characters and uneven spacing to bleed-through from the back of the page—and any number of random inconsistencies.

The goal is to get the main text into a clean .txt file so we can move on to grammar cleanup.

Grammatic Tweaking

There are always going to be issues with the raw OCR output. These can be anything from a missed space or misplaced punctuation to completely misread characters. It's a common problem with OCR and usually easier to fix than the actual training.

The idea here would be to send the text through a grammar-checking tool—like Grammarly or ChatGPT—using a prompt that corrects the worst of the issues. This would be done through an API (Application Programming Interface), running each file through a script that calls the model and returns a cleaner version of the text.

Rebuilding

Once the text is cleaned up and easy to read, the final step is to put everything back together in order—rebuilding the book into a Word document or PDF. This part should be relatively simple. As long as the pages are processed in order, or each one is fully handled before moving on to the next, the output should be consistent.

When it's done, you'll have a full, digitized version of *CIRCVNCISION DE COMEDIAS*, or any other book with similar formatting.

Previous Knowledge

I didn't come into this project with a lot of foundational knowledge. I already knew Python and was comfortable using Linux. My experience with Linux ended up being a big help, especially since the project itself didn't necessarily require much Python. That's because I chose to work in a containerized environment.

That decision led to my first big obstacle. I had used containers before, but only under supervision—never having to build or configure one myself. After some research, I found that training inside a container would be the best way to maintain progress and experiment with different models without constantly resetting my setup.

I was already familiar with VS Code from my computer science studies, so working in that editor felt natural. Another piece of knowledge that came in handy was GitHub. I had used it several

times in the past for turning in assignments and working on personal projects, so I wasn't starting from scratch there either.

However, when it came to the actual tools I would need for this project, I was completely new. I had never worked with OCR before and had no idea what .tiff, .box, or .lstmf files were.

In short, I had next to no idea how to complete this project at the start.

Strategy and Problems

There were many pivots in my attempt at this project. Initially, Prof. Puente Agueda suggested that I use Tesseract OCR. After completing some research, I decided to try EasyOCR instead, as it seemed simpler for beginners. The drawback, however, was that it was much less effective for training your own model.

After about a week of work, I managed to get an EasyOCR model to train, none of which was effective. In fact, it couldn't correctly recognize even a single word. After hitting this wall, I turned back to Prof. Puente Agueda's original advice and committed to using Tesseract.

Although Tesseract is definitely the right tool for achieving our goals, what I had learned from working with EasyOCR had prepared me for this more advanced and customizable system.

The next strategic decision I faced was whether to use .box files. At that time, I was testing specific characters and came across some "anti-box file training" enthusiasts. It seemed like there

was a lot of support for this free-form method. I thought that, given the difficulty of labeling all text positions, if the program could transcribe without breaking the problem into smaller steps, we might eliminate the need for manual box annotation.

Although well-intentioned, the anti-box method proved to be extremely buggy and difficult to train. After switching back to .box files for just a day, I realized how inconsistent and unreliable the process was without them.

My next major issue came during the generation of .box and .lstmf files. Sometimes the .box files would work, but the .lstmf files would fail to generate. Eventually, I discovered the issue was with my .tiff images. Possibly corrupted, they had originally come from screenshots. I re-exported them and simplified their format, which resolved the generation issues.

The final major challenge was the language configuration. When I tried to train Tesseract, I kept encountering errors like “404: Language not found” or messages about version incompatibility. I eventually realized that Tesseract was looking for English language training data, but I had only downloaded Spanish in my Dockerfile. After updating the Dockerfile to include both English and Spanish, I was able to proceed smoothly through the training process.

Iterations

Throughout the process of training my model, I went through three major phases.

Initially, I was training with specific characters, mainly as a way to learn how to create a custom-trained model using Tesseract. This early model was extremely ineffective. While it became fairly good at identifying individual characters, it performed poorly with any combination of letters beyond that. Since I believed this version was essentially useless, I decided not to save a GitHub branch for it.

In the next phase, I created a model trained solely on words. I saved this version in a GitHub branch called TrainingDataWordsOnly. As the name suggests, this model used words rather than individual characters or full sentences for training. The results were surprisingly strong when tested on sentences. The model performed with high accuracy. However, once I tried feeding it full paragraphs, performance dropped significantly, and accuracy became extremely poor.

To address this, the third version (saved on the main branch of the Git repository) used a mix of words and full sentences for training. Although this model was trained on more data, it surprisingly performed worse on sentence-level OCR than the previous version. This unexpected result had some hidden advantages. The model now handled paragraphs with similar accuracy as

sentences and, although it made more errors, most were limited to punctuation and minor character issues.

Despite the results appearing unreadable at first glance, this version marked a real breakthrough. After testing the output using my personal ChatGPT account, I found that the errors were easily correctable. GPT had no trouble fixing grammatical issues and minor character misreads from the model's output. Because of this level of accuracy, future iterations could include even more words, using the previous model's output as reliable training data for the next. This means that, once a post-processing correction step is integrated, the system's final output could be made almost entirely accurate.

Future Additions

Currently, we have a working model trained using both words and sentences as input data. I see the next steps unfolding as follows:

- Connect ChatGPT via API to grammatically correct the model's output files.
- Use these corrected files to generate additional training data and improve model accuracy.
- Create a program to deconstruct the scanned book into its structural components—margins, titles, handwritten notes, and main paragraphs.
- Finally, build a tool that compiles all components into a formatted Word document.

Once these steps are completed, I see no reason why the entire book couldn't be processed end-to-end using this single program. The format would also be highly adaptable, capable of working with nearly any type of text, as long as some initial training data is available.

The specific next step is connecting the API. I began researching how to do this but ran out of time before the end of my research term. Based on my findings, this shouldn't be a particularly

difficult task and could likely be completed in a single productive day. However, it does require access to a ChatGPT account (or similar service), which typically means a subscription.

By far, the most technically challenging part of the project will be breaking down the book's PDF into segments based on writing style. While this seems related to OCR, it may actually be easier or similarly complex, given that .box files are generally straightforward to use. That said, I did encounter some challenges in this area that would need to be addressed in future work.

Installing VS Code

VS Code is a code editor app. To install VS Code, one should go to their website to follow their specific and updated instructions.

The link is here: https://code.visualstudio.com/docs/setup/setup-overview

VS Code is a free code editor, which runs on the macOS, Linux, and Windows operating systems. Getting up and running with Visual Studio Code is quick and easy. It is a small download so you can install in a matter of minutes and give VS Code a try.

VS Code is lightweight and should run on most available hardware and platform versions. You can review the System Requirements to check if your computer configuration is supported.

Download and install Visual Studio Code for your platform

- macOS
- Linux
- Windows

Note

VS Code ships monthly releases and supports auto-update when a new release is available.

Install additional components

Install Git, Node.js, TypeScript, language runtimes, and more.

© Visual Studio Code

Docker Desktop

Docker is an operating system for containers. To install Docker Desktop, one should go to their website to follow their specific and updated instructions.

The link is here: https://docs.docker.com/get-started/introduction/get-docker-desktop/

Docker Desktop is the all-in-one package to build images, run containers, and so much more. This guide will walk you through the installation process, enabling you to experience Docker Desktop firsthand.

Apple silicon Install Instructions

Intel Install Instructions

Windows Install Instructions

Linux Install Instructions

Once it's installed, complete the setup process and you're all set to run a Docker container.

© Docker Inc.

Setting Up Environments

Opening Docker Desktop

Open Docker Desktop and agree to its conditions.

Pulling From GitHub

To open a VS Code project with all the same files as the version on Git Hub youll have to pull it from the repository.

- First, you'll need to go to the repository with this link: https://github.com/Udog-ILLINOIS/oldBookTesseractOCRtranscription.git

- Then navigate to the code dropdown button, to https tab, and copy that link to your clipboard

- Then you'll need to open VS Code, then you'll need to navigate to the source control (the circle branched icon)

- After that you'll need to look under the Start prompt on the page and click on Clone Git Repository

- Once that brings you to an open search bar at the top of the page, paste the link you copied from git and press enter

- You'll be asked where you want to save these files. Create a folder for this project and select as the destination

- You should then see all the files from the repository populate the project

Docker File

Here is the exact code of my dockerfile with comments for clarity:

---- START CODE ---

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV TESSDATA_PREFIX=/usr/local/share/tessdata

RUN apt update && apt install -y \
  git curl wget make cmake g++ pkg-config \
  libleptonica-dev \
  libtiff-dev libjpeg-dev zlib1g-dev \
  libpng-dev libicu-dev \
  libpango1.0-dev libcairo2-dev \
  libtool automake autoconf \
  fonts-dejavu fonts-liberation fonts-freefont-ttf \
```

```
    && apt clean

# Build Tesseract with training on
WORKDIR /opt
RUN git clone --branch 4.1.1 https://github.com/tesseract-ocr/tesseract.git && \
    cd tesseract && \
    ./autogen.sh && \
    ./configure --enable-training && \
    make -j$(nproc) && \
    make training && \
    make install && \
    make training-install && \
    ldconfig

# Download Spanish and English traineddata
RUN mkdir -p /usr/local/share/tessdata && \
    wget -O /usr/local/share/tessdata/spa.traineddata \
    https://github.com/tesseract-ocr/tessdata_best/raw/main/spa.traineddata && \
    wget -O /usr/local/share/tessdata/eng.traineddata \
    https://github.com/tesseract-ocr/tessdata_best/raw/main/eng.traineddata

WORKDIR /workspace

### ---- END CODE --- ###
```

Running Container

In order to run this container, you will need to open the terminal on VS Code. In order to do this easily, look at the top right of the VS Code window. You'll see three boxes, Two with vertical rectangles and one with a horizontal rectangle. Press the one with the horizonal rectangle. From there, you'll need to paste this code into your terminal.

First:

```
docker build -t tesseract-training .
```

Second, note this path i.e. the /Users/yourUser/Desktop... is unuiqe to your position. To get your path, left click on the Dockerfile in the Explorer tab on the left of your VS Coder terminal. Copy the path. When you paste that path, everything except for the /Dockerfile at the end is your path:

```
docker run -it --rm -v /Users/yourUser/Desktop/nameOfYourFolder/anyOtherInsudeFolder:/workspace
tesseract-training bash
```

Third (this is for later usage and should have been in the Dockerfile.... sorry):

```
apt update && apt install -y imagemagick
```

Converting file types and Creating .gt.txt and .txt Files

To train the model you'll need data, images to train on. I already have these images in the project but if you wish to add more, take pictures of the portion you want to add, and name it something to identify this image over the rest. Then you'll need to convert this file to a .tiff. I like to go to cloud convert and upload my file and then download the .tiff file. Here is the link for cloud convert: https://cloudconvert.com/png-to-tiff

Once you have these .tiff files you should upload them to their corresponding place, i.e. sentences, words, paragraphs etc. in the trainingData folder.

You'll also need to create .gt.txt and .txt files. This is easy, just add a file and name it (the name of the .tiff file).gt.txt and another as (the name of the .tiff file).txt. In these files they should contain the same things, the true transcription of the image that corresponds to them. Notheing more.

Creating .box Files

Once you're ready for this step, go back to the VS Code terminal and paste this commands:

```
find /workspace/trainingData -type f -name '*.tiff' | while read -r img; do
  filename=$(basename "$img")
  dir=$(dirname "$img")
  base="${filename%.tiff}"

  # Skip old versions
  [[ "$base" == old-* ]] && continue

  echo "Generating box file for $img"
  tesseract "$img" "$dir/$base" --psm 6 -l spa batch.nochop makebox
done
```

Creating .lstmf Files

After the .box files have been successfully created, paste this into the terminal:

```
find /workspace/trainingData -type f -name '*.tiff' | while read -r img; do
  filename=$(basename "$img")
  dir=$(dirname "$img")
  base="${filename%.tiff}"

  # Skip old versions
  [[ "$base" == old-* ]] && continue

  box_file="$dir/$base.box"
  gt_file="$dir/$base.gt.txt"

  if [[ -f "$box_file" && -f "$gt_file" ]]; then
    echo "Generating .lstmf for $img"
    tesseract "$img" "$dir/$base" --psm 6 -l spa lstm.train
  else
    echo "Skipping $img: missing .box or .gt.txt"
```

```
  fi
Done
```

Training The Model

Once you have your .tiff, .gt.txt, .box, and .lstmf files you are ready to train. To train paste these commands in terminal at this order:

First:

```
find /workspace/trainingData -type f -name '*.lstmf' > /workspace/trainingData/training_files.txt
```

Second:

```
mkdir -p /workspace/output
```

Third:

```
lstmtraining \
  --model_output /workspace/output/spa_custom \
  --traineddata /usr/local/share/tessdata/spa.traineddata \
  --continue_from /workspace/spa.lstm \
  --train_listfile /workspace/trainingData/training_files.txt \
  --max_iterations 1000
```

Fourth:

```
lstmtraining \
  --stop_training \
  --continue_from /workspace/output/spa_custom_checkpoint \
  --traineddata /usr/local/share/tessdata/spa.traineddata \
  --model_output /workspace/output/spa_custom.traineddata
```

Fifth:

```
export TESSDATA_PREFIX=/workspace/output
```

Running New Images

To test and run new images, you should follow these steps.

First the image you mean to test must be inside /trainingData/practicePNGS

Once that is true you can run this command (Note YOURIMAGE should be the name of the image):

```
tesseract /workspace/trainingData/practicePNGS/YOURIMAGE.png output_result -l spa_custom
```

The output will be in the main folder under the name output_result. If you run any more test images without changing the name of the output file, they will overwrite each other.

Once you have your transcription, I encourage you to run it through ChatGPT and ask it to fix the grammar of this old Spanish OCR transcribed paragraph.

**** IF YOU WANT TO EDIT THIS REPOSITORY EMAIL ME AT ewanjm2@illinois.edu WITH YOUR GITHUB USERNAME AND I'LL ADD YOU AS A CALBORATOR****

Congrats on your transcription!